Guía de Estilo de Código – EcoHarmony Park

1. Introducción

Acá dejamos escritas las reglas con las que vamos a programar en el grupo para EcoHarmony Park.

Para qué sirve. Ponernos de acuerdo en cómo nombrar las cosas, cómo formatear, cómo comentar, cómo diseñar y cómo escribir las pruebas.

Hasta dónde llega. Vale para todo el código y las pruebas de la User Story *Inscribirme a actividad*, que vive en la carpeta `Tp6/Programa`.

Cómo leerlo. Lo que dice «debe» es obligatorio; lo que dice «conviene» es recomendado. Si aparece algo que no está cubierto, lo charlamos en el grupo, lo agregamos acá y lo anotamos en el historial de cambios (sección 10).

2. Lo que decidimos como grupo

Arrancamos con la guía oficial de Python (PEP 8) y le sumamos nuestras propias reglas.

Qué decidimos	Lo que elegimos	Por qué
Lenguaje y versión	Python 3.14 (verificado con CPython 3.14.8)	Es simple de leer, lo conocemos y es la versión con la que corre Render
Framework web	Django	Trae modelos, formularios, admin y migraciones listos
Framework de pruebas	pytest + pytest-django	Menos código que unittest, y permite probar varios casos de datos con parametrize
Para simular cosas	unittest.mock	Nos deja "hacer de cuenta" que existe la API de Brevo o el servidor de correo
Formato automático	Black	Formatea solo, así no discutimos por espacios y comillas
Chequeo de estilo	Ruff	Es rápido, avisa de imports sin usar, nombres mal puestos y errores comunes
Tipos	Type hints + mypy	Detecta errores antes de correr el código y documenta qué recibe cada función

Qué decidimos	Lo que elegimos	Por qué
Entorno y dependencias	venv + pip con requirements.txt y requirements-dev.txt	Todos trabajamos con las mismas versiones (fijadas)
Idioma del código	Español para el dominio, inglés para lo técnico	Los términos del negocio (Actividad, Horario, Inscripcion) son los mismos del enunciado

3. Cómo nombramos las cosas

Un nombre tiene que decir para qué sirve lo que nombra, con las palabras del enunciado. Nada de abreviaturas raras (cant, tmp, x): si nadie las entiende, no van.

Qué	Cómo se escribe	Ejemplo
Clases	PascalCase, un sustantivo	Actividad, Horario, DatosParticipante
Funciones y métodos	snake_case, con un verbo	validar_solicitud(), registrar_inscripcion()
Variables y parámetros	snake_case, un sustantivo	participantes, fecha_de_visita
Constantes	MAYUSCULAS_CON_GUION_BAJO, a nivel de módulo	EDAD_MINIMA_TIROLESA
Módulos y paquetes	minúscula y snake_case	validacion_edad.py, inscripciones.py
Booleanos	Arrancan con es_, tiene_, puede_ o admite_	es_menor_de_edad, admite_inscripciones()
Atributos internos	Guion bajo adelante	_validar_participante(), _enviar_por_brevo()
Archivos y funciones de prueba	test_ + lo que prueban	test_inscripciones.py
Excepciones	Terminan en Error y dicen qué pasó	CupoInsuficienteError

Otras reglas

- Si una función devuelve un valor, que el nombre no prometa otra cosa (no uses `obtener_` si en realidad calcula).
- Las colecciones van en plural: participantes, horarios.
- No mezclamos idiomas en un mismo nombre (calcular_total_price no vale).
- No usamos nombres que tapen los de Python (list, id, type, input).

4. Formato y orden del código

El formato lo controlan Black y Ruff por nosotros. Si el chequeo falla, el código no se mete en `main`, y listo.

Tema	Regla
Indentación	4 espacios, nada de tabulaciones
Una cosa por línea	Una instrucción por línea, sin amontonar
Líneas en blanco	Las que deja Black: dos entre funciones y clases, una entre métodos
Imports	Sin comodines (*), en orden alfabético y sin los que no se usan

Cómo organizamos las carpetas

```
Programa/
    config/    settings.py, urls.py
    core/      models.py, views.py, inscripciones.py, validacion_edad.py, email.py
        templates/core/, static/core/
        management/commands/  cargar_actividades.py, cargar_horarios.py
        tests/  test_inscripciones.py, test_validacion_edad.py, ...
```

5. Comentarios y documentación

El código tiene que entenderse solo. Comentamos únicamente lo que no se ve a simple vista.

- **Docstring corto:** cada módulo, clase y función pública llevará una frase que explique para qué sirve.
- **Comentarios con #:** solo para aclarar el *porqué* de una regla, por ejemplo: «Render bloquea el SMTP saliente en el plan gratuito».
- **Nada de código viejo comentado:** si no se usa, se borra (para eso está Git).
- **Pendientes:** se marcan con `TODO` y se sacan antes de entregar.

Ejemplo

```
def validar_edad_para_actividad(actividad: str, edad: object) -> None:
    """Valida que la edad del visitante permita realizar la actividad."""
```

6. Buenas prácticas de diseño

Estas reglas guían el paso *Refactor* del ciclo TDD y son la base del documento de justificación de decisiones de diseño.

1. **Responsabilidad única:** cada módulo tiene una razón para cambiar (validar la edad, registrar la inscripción y enviar el correo están en lugares distintos).

2. **Sin números ni textos mágicos:** las reglas del enunciado van en constantes con nombre, como EDAD_MINIMA_TIROLESA = 8.
3. **Lo externo, aislado:** el envío por Brevo y por SMTP vive solo en core/email.py, para poder simularlo en las pruebas sin tocar el resto.
4. **Validar temprano:** los datos inválidos (DNI mal escrito, edad menor a la mínima, horario que ya cerró) se rechazan al inicio con una excepción específica.
5. **Excepciones con significado:** no se capturan Exception genéricas ni se devuelve None para avisar un error; se usan excepciones propias que heredan de ValueError.
6. **Inmutabilidad:** los objetos de valor (DatosParticipante) no cambian después de crearse (dataclass con frozen=True).
7. **Sin duplicación:** si una regla aparece dos veces, se extrae a un único lugar (DRY).
8. **Sin efectos ocultos:** un método de consulta (cupos_disponibles()) no modifica el estado.

Ejemplo de regla de negocio como constante

```
EDAD_MINIMA_TIROLESA = 8

EDADES_MINIMAS_POR_ACTIVIDAD = {
    "TIROLESA": EDAD_MINIMA_TIROLESA,
    ...
}
```

7. Convenciones para las pruebas (TDD)

Cada funcionalidad nace de una prueba que falla; el código de producción se escribe solo para hacerla pasar.

Fase	Qué se hace	Regla de estilo
Red	Escribir una prueba que falle	La prueba falla por la razón esperada, no por un error de import o de sintaxis
Green	Escribir el código mínimo para que pase	Se admite código simple, incluso duplicado, si hace pasar la prueba
Refactor	Mejorar el diseño sin romper las pruebas	Todas las pruebas siguen en verde y se aplican las reglas de la sección 6

Reglas simples para escribir pruebas

- **El nombre cuenta qué se prueba.** Por ejemplo: `test_rechaza_inscripcion_sin_aceptar_terminos`. Con leerlo ya se entiende.
- **Cada prueba tiene tres partes,** separadas por una línea en blanco: preparar lo que necesito, hacer la acción y comprobar el resultado.

- **Una prueba, una cosa.** Si falla, tiene que ser obvio por qué.
- **Sin `if` ni bucles** dentro de la prueba. Si hay que probar varios valores, usamos `@pytest.mark.parametrize`.
- **Cada prueba se arma sola.** No depende de otra ni del orden en que se corran; lo que se repite va como fixture en `conftest.py`.
- **Lo de afuera se simula.** Brevo y el servidor SMTP no se llaman de verdad en las pruebas; usamos `mock`.

Qué casos probamos siempre

- Que ande lo normal (por ejemplo, inscribir dos participantes en la tirolesa).
- Los límites: la edad mínima justa y un año menos, el último cupo libre y uno de más.
- Los datos mal cargados (DNI inválido, correo inválido, términos sin aceptar).
- Qué pasa si falla algo externo (por ejemplo, Brevo no responde y la inscripción se guarda igual).

8. Control de versiones

Usamos Git para guardar el trabajo del grupo. Si lo hacemos prolijo, el historial muestra cómo fuimos armando el código paso a paso, y eso es justo lo que el TP quiere ver en TDD.

Las reglas, simples

1. `main` **es sagrada.** Ahí solo va código que anda.
2. **Las tareas largas, en su rama.** Por ejemplo `feature/tp6-envio-correo`; se integran a `main` cuando están terminadas.
3. **Guardamos seguido y en pedacitos.** Un commit por cada paso del ciclo: uno cuando escribimos la prueba que falla, otro cuando la hacemos pasar y otro cuando ordenamos el código.
4. **El mensaje cuenta qué hiciste.** Empieza con el tipo (`FEAT:`, `TEST:`, `Docs:` o `Especial:`); en los pasos TDD se agrega la fase (`(RED)`, `(GREEN)` o `(REFACTOR)`). Sigue con una frase corta en español, en infinitivo y en una línea.
5. **Antes de pasar a `main`, se prueba todo.** Corremos `pytest`, `black --check .`, `ruff check .` y `mypy .` (la lista completa está en el README de Programa). Si algo falla, se arregla primero.
6. **Que lo mire otro.** Un compañero revisa el cambio con la checklist de la sección 9 antes de que entre a `main`.
7. **No subimos basura.** El entorno virtual (`.venv/`), las cachés (`__pycache__/`, `.pytest_cache/`), las bases SQLite y los `.env` van en el `.gitignore`.

Cómo se ve en la práctica

```
FEAT: (RED) Adjuntar el QR al correo de Brevo
FEAT: (GREEN) Adjuntar el QR al correo de Brevo
FEAT: (REFACTOR) Separar el envio por Brevo y por SMTP
```

9. Checklist de revisión y referencias

Checklist antes de integrar un cambio

- ☐ Existe una prueba que fallaba antes del código (fase Red) y ahora pasa
- ☐ Todas las pruebas pasan y cubren camino feliz, límites y errores
- ☐ Black, Ruff y mypy no reportan nada
- ☐ Los nombres siguen la sección 3 y usan el vocabulario del dominio
- ☐ No hay números mágicos, código comentado ni duplicación
- ☐ Las clases y funciones públicas tienen docstring
- ☐ Los commits reflejan las fases Red, Green y Refactor

10. Historial de cambios

Fecha	Cambio
15/10/2026	Versión inicial
27/10/2026	Se ajusta a lo que usa el TP6: Python 3.14, Django, Black/Ruff/mypy en lugar de Checkstyle, docstrings en lugar de Javadoc, ejemplos de *Inscribirme a actividad*, carpetas reales y formato de commits del grupo